

Table of Contents

1	B.Tech Final Year Project Documentation (Start-to-End)	3
1.1	Project Title	3
1.2	College, Student, and Guide Details	3
1.3	Academic Session.....	3
1.4	Certificate	4
1.5	Declaration.....	4
1.6	Acknowledgement	4
1.7	What this document is for	4
1.8	1. Abstract.....	5
1.9	2. Problem Statement.....	5
1.10	3. Objectives.....	5
1.11	3.1 Project Methodology (End-to-End Approach)	6
1.12	4. Tech Stack Used (as implemented in the repository)	7
1.12.1	4.1 AI / Computer Vision (Python)	7
1.12.2	4.2 Backend (Node.js / Express)	7
1.12.3	4.3 Frontend (React + Vite).....	7
1.13	4.1 System Requirements (Practical)	8
1.13.1	AI Server Requirements (FastAPI + YOLO)	8
1.13.2	Backend Requirements (Node.js + MongoDB)	8
1.13.3	Frontend Requirements (React + Vite)	8
1.14	5. System Architecture (Overall)	8
1.15	6. Repository Structure (important folders)	9
1.16	7. AI Model (YOLOv8) Explanation.....	9
1.16.1	7.1 Why YOLOv8?	9
1.16.2	7.2 What your repository uses	9
1.17	8. Dataset Processing (for training)	10
1.18	9. AI Inference Server (FastAPI).....	10
1.18.1	9.1 How the server loads the YOLO model.....	10
1.18.2	9.2 The detection endpoint: POST /detect	10
1.19	10. Gradio Demo (Optional Visualization)	11
1.20	11. Backend System (Express + MongoDB).....	12

1.20.1	11.1 Backend entry point	12
1.20.2	11.2 Security, CORS, and API Documentation (What the backend does)	12
1.21	12. Backend Routes (APIs) - Full Explanation.....	13
1.21.1	12.1 Authentication APIs.....	13
1.21.2	12.2 JWT Middleware	13
1.21.3	12.3 Issue APIs (Citizen reporting + listing + AI detect).....	14
1.22	13. Backend Comment APIs.....	16
1.23	14. Backend Voting APIs	16
1.24	15. Backend Attachments APIs (Upload images/files).....	17
1.24.1	15.1 POST /api/issues/:id/attachments	17
1.24.2	15.2 GET /api/attachments/:id/signed-url.....	17
1.25	16. MongoDB Data Models.....	17
1.25.1	16.1 User	17
1.25.2	16.2 Issue	17
1.25.3	16.3 Comment.....	18
1.25.4	16.4 Vote	18
1.25.5	16.5 Attachment	18
1.25.6	16.6 IssueHistory	18
1.26	17. Backend Validation and Error Handling	18
1.26.1	17.1 Joi validation.....	18
1.26.2	17.2 Error handling middleware.....	18
1.27	18. Scripts and Tests	18
1.27.1	18.1 Seed script	18
1.27.2	18.2 Clear script.....	19
1.27.3	18.3 Test.....	19
1.28	19. Frontend - Citizen Portal (Home).....	19
1.28.1	19.1 Key idea	19
1.28.2	19.2 Token handling	19
1.28.3	19.3 Main app flow	19
1.28.4	19.4 Citizen reporting UI (code walkthrough)	20
1.28.5	19.5 Issues list and voting.....	21
1.28.6	19.6 Profile editing.....	22
1.29	20. Frontend - Admin Portal	22

1.29.1	20.1 Admin authentication	22
1.29.2	20.2 Admin dashboard	22
1.29.3	20.3 Issue map (admin)	22
1.29.4	20.4 Issue detail modal (admin)	22
1.30	21. Deployment / Running the Whole System	23
1.30.1	21.1 Local setup (high-level)	23
1.30.2	21.2 Render deployment (as defined in repo).....	24
1.30.3	21.3 Backend integration guide (already present in repo)	24
1.31	22. Expected Workflow (From User Perspective).....	25
1.31.1	Step A: User Registration/Login	25
1.31.2	Step B: Reporting an Issue (with AI detection).....	25
1.31.3	Step C: Viewing Issues and Voting	25
1.31.4	Step D: Admin Management.....	25
1.32	23. Limitations (Based on Current Code).....	26
1.33	24. Conclusion.....	26
1.34	25. Future Scope (From your repo + common improvements).....	26
1.35	References	27
1.36	Appendix A: Important Code Files	27

1 B.Tech Final Year Project Documentation (Start-to-End)

1.1 Project Title

Snap2Solve: AI-powered Smart City Issue Detection and Reporting System

1.2 College, Student, and Guide Details

- **Institute (College):** JSPM University, Pune
- **Department:** B.Tech CSE
- **Student Name:** Shree Joshi,Saloni Jadhav, Shravani Gaikwad, Vaishnavi kadam
- **Guide Name:** Mrs. Punam Rahangdale

1.3 Academic Session

2025-2026

1.4 Certificate

This is to certify that **Shree Joshi**, Saloni Jadhav, Shravani Gaikwad, Vaishnavi kadam has successfully completed the Final Year Project titled: **“Snap2Solve: AI-powered Smart City Issue Detection and Reporting System”** under the guidance of **Mrs. Punam Rahangdale**, in partial fulfillment of the requirement for the degree of **B.Tech (CSE)** at **JSPM University, Pune**.

(Signature of Guide / Coordinator)

(Signature of Student)

1.5 Declaration

I hereby declare that the project work titled: **“Snap2Solve: AI-powered Smart City Issue Detection and Reporting System”** submitted for B.Tech (CSE) at JSPM University, Pune is an original work done by me. Any references and resources used in this project are appropriately cited in the document.

(Date)

(Place: Pune)

(Signature of Student)

1.6 Acknowledgement

I would like to express my sincere gratitude to my guide **Mrs. Punam Rahangdale** for continuous support and valuable guidance throughout this project. I also thank my college for providing the necessary learning environment and resources.

1.7 What this document is for

This document explains your complete project **from the beginning to the end**, in beginner-friendly language. It is based on the code present in this repository:

- **AI detection (YOLOv8 + Gradio demo):** model/
 - **AI detection server (FastAPI used by the main system):** model/api.py
 - **CivicFix backend (Express + MongoDB + JWT + file uploads):**
frontback_end/backend/
 - **Citizen (home) front-end (React):** frontback_end/frontend/home/
 - **Admin front-end (React):** frontback_end/frontend/admin/
-

1.8 1. Abstract

Smart cities require fast identification and resolution of infrastructure problems such as potholes and garbage accumulation. Manual reporting is slow and inconsistent. This project implements an automated system that:

1. Uses a **custom YOLOv8 object detection model** to detect issues from images.
 2. Provides an interactive **AI detection server** that returns predictions and confidence scores.
 3. Integrates the AI with a **full-stack web application** where citizens can register/login, submit an issue with an image and location, and track the status.
 4. Provides an **admin portal** to manage issues, update statuses, and view reports.
-

1.9 2. Problem Statement

Urban infrastructure issues negatively impact safety and quality of life. Citizens often report problems manually, which leads to delays and poor data quality.

This project automates the process by combining: - Computer vision detection (YOLOv8) - A REST API backend (Express + MongoDB) - Web user interfaces for citizens and administrators

1.10 3. Objectives

1. Train or use a custom **YOLOv8** model for detecting smart-city issues from images.
 2. Deploy an **AI inference server** (FastAPI) that can be called by the backend.
 3. Build a backend system that supports:
 - User authentication with JWT
 - Issue creation, listing, updating, and deletion
 - Comments and voting
 - File uploads (local disk and optional S3 support)
 4. Build a citizen portal to:
 - Upload/capture images
 - Automatically run AI detection to help fill category labels
 - Submit issues with geo-location
 5. Build an admin portal to:
 - View and manage issues
 - Update issue status
 - View comments and issue details
-

1.11 3.1 Project Methodology (End-to-End Approach)

Your project was built in the following stages:

1. Problem Understanding

- Identify the types of civic problems to detect (pothole, garbage, etc.).
- Understand the reporting workflow needed by citizens and administrators.

2. Dataset Preparation (Roboflow workflow)

- Create bounding box annotations for each class.
- Apply augmentation (flip, rotate, brightness changes, blur, etc.).
- Split data into train/validation/test sets.

3. Model Training / Selection (YOLOv8)

- Train a YOLOv8 model (YOLOv8m / custom weights as available in the repo).
- Evaluate and select the final weights file (for inference: best.pt).

4. AI Inference Service

- Create a FastAPI server that loads YOLO weights and exposes a /detect endpoint.
- Implement logic to return:
 - top predicted label + confidence
 - full list of detected labels/confidences
 - department mapping from label

5. Backend API (CivicFix)

- Implement user registration/login using JWT.
- Implement issue CRUD operations with validation using Joi.
- Implement image upload using multer:
 - local disk mode for development
 - optional S3 mode for deployment
- Implement an AI integration route:
 - backend receives image -> calls AI server -> returns prediction to UI

6. Frontend UI (Citizen + Admin)

- Citizen portal:
 - capture/upload image
 - call /issues/detect
 - use AI results to auto-fill category/labels
 - submit issue with image + geo-location
- Admin portal:
 - view issue list
 - open issue details
 - update issue status
 - view comments

7. Testing and Deployment

- Use `render.yaml` to deploy the AI server, backend, and frontends together.
 - Use backend health endpoints and basic tests (jest) for validation.
-

1.12 4. Tech Stack Used (as implemented in the repository)

1.12.1 4.1 AI / Computer Vision (Python)

- ultralytics (YOLOv8 inference)
- fastapi (AI detection server)
- uvicorn (ASGI server for FastAPI)
- gradio (demo UI for image-based detection)
- pillow (image conversion for Gradio output)
- python-multipart (FastAPI file uploads)
- Standard Python libs: `os`, `shutil`, `threading`, `contextlib`

1.12.2 4.2 Backend (Node.js / Express)

- express (REST API server)
- cors (cross-origin requests)
- dotenv (environment variables)
- helmet (security headers)
- express-mongo-sanitize (reduce injection risk for MongoDB)
- mongoose (MongoDB ODM)
- multer (multipart/form-data file upload handling)
- jsonwebtoken (JWT verification)
- bcryptjs (password hashing)
- axios (calling the AI server)
- joi (request validation schemas)
- aws-sdk (optional S3 upload + signed URLs)
- swagger-jsdoc + swagger-ui-express (API documentation)
- jest + supertest (basic tests)

1.12.3 4.3 Frontend (React + Vite)

- React, ReactDOM
- Vite for building and running
- UI libraries (Radix UI components + lucide-react icons + Tailwind utility styling)
- sonner for toast notifications
- recharts for charts (admin dashboard)
- date-fns for date formatting (admin downloads)

1.13 4.1 System Requirements (Practical)

1.13.1 AI Server Requirements (FastAPI + YOLO)

- Python 3.9+ (the repo uses modern FastAPI and Ultralytics)
- Trained YOLO weights file:
 - `model/best.pt` (expected by `model/api.py` and `model/app.py`)
- Storage for temporary uploaded images:
 - `model/uploads` (created automatically by the server code)
- A machine that can run YOLO inference:
 - GPU is recommended for faster inference, CPU still works but slower.

1.13.2 Backend Requirements (Node.js + MongoDB)

- Node.js ≥ 16
- MongoDB (local or cloud) with a connection string in `MONGODB_URI`
- Environment variables for security and integration:
 - `JWT_SECRET`, `AI_SERVER_URL`, etc.

1.13.3 Frontend Requirements (React + Vite)

- Modern web browser
- API URL configured via Vite environment variable:
 - `VITE_API_URL`

1.14 5. System Architecture (Overall)

At runtime, the system works like this:

1. **Citizen UI** (Home Frontend)
 - User uploads/captures an image
 - User submits the image for AI detection
 - UI receives AI result: predicted label + confidence + department mapping
 - UI creates an issue in the backend (with image, category, labels, and location)
2. **Backend API** (Express + MongoDB)
 - Receives the citizen request
 - Validates request using Joi schemas
 - Authenticates using JWT middleware (where required)
 - Stores issue data in MongoDB
 - Calls the AI server when user requests `/issues/detect`
3. **AI Server** (FastAPI + YOLOv8)

- Loads YOLO model (best.pt)
- Receives image upload at /detect
- Runs YOLO inference and returns:
 - prediction (top label)
 - confidence
 - department (mapped from label)
 - all_detections (list of all detected boxes with labels + confidences)

4. Admin UI

- Uses backend APIs to list issues
- Displays issue details, comments, and allows status updates

1.15 6. Repository Structure (important folders)

Your project is split into three major parts:

- model/
 - YOLO model inference + dataset processing scripts
 - AI detection server used by the main system
 - Optional Gradio demo
- frontback_end/backend/
 - Express API server + MongoDB models + routes + middleware
- frontback_end/frontend/home/
 - Citizen portal UI
- frontback_end/frontend/admin/
 - Admin portal UI

Additional deployment configuration: - render.yaml (shows how the parts are deployed together)

1.16 7. AI Model (YOLOv8) Explanation

1.16.1 7.1 Why YOLOv8?

YOLO (You Only Look Once) is a popular real-time object detection model. YOLOv8 returns bounding boxes for detected objects along with: - class label - confidence score

1.16.2 7.2 What your repository uses

Two main inference implementations exist:

1. **Gradio demo:** model/app.py

2. FastAPI inference server: model/api.py

In production-like deployment, render.yaml starts the AI server using: uvicorn
api:app ... from model/ directory.

1.17 8. Dataset Processing (for training)

Your repository includes a script: - model/process_local_datasets.py

It merges multiple downloaded/renamed datasets and **remaps class IDs** into the final class order: 0. Pothole 1. Garbage 2. Vandalism 3. Streetlight

It also creates a data.yaml file for YOLO training pointing to: - train/images, valid/images, test/images - nc (number of classes) - names (class names in order)

1.18 9. AI Inference Server (FastAPI)

Main file: model/api.py

1.18.1 9.1 How the server loads the YOLO model

Instead of loading the model at startup and blocking the server, your code: - starts a background thread (load_model_task) - sets global variables: - model - is_loading

Then it exposes health endpoints: - / (server running) - /health (model_loaded + is_loading)

1.18.2 9.2 The detection endpoint: POST /detect

Your endpoint receives: - file: UploadFile

It saves the file into: - UPLOAD_FOLDER = "uploads"

Then it runs: - results = model(file_path)

Finally it parses r.boxes: - reads box.cls (class index) - reads box.conf (confidence) - maps class index -> class name using model.names

It chooses the highest confidence label as top_prediction. Then it maps top_prediction to a department using department_map.

It returns JSON: - prediction - confidence - department - all_detections (list of {label, confidence})

1.18.2.1 Key code (AI server)

```
def load_model_task():  
    global model, is_loading  
    is_loading = True
```

```

...
model = YOLO(MODEL_PATH)
...
is_loading = False

@app.post("/detect")
async def detect(file: UploadFile = File(...)):
    ...
    file_path = os.path.join(UPLOAD_FOLDER, file.filename)
    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    results = model(file_path)

    detections = []
    top_prediction = "unknown"
    confidence = 0

    for r in results:
        for box in r.bboxes:
            cls = int(box.cls)
            label = model.names[cls]
            conf = float(box.conf)
            detections.append({"label": label, "confidence": round(conf, 2)})
            if conf > confidence:
                top_prediction = label
                confidence = conf

    department = department_map.get(top_prediction, "Municipal Department")
    return {
        "prediction": top_prediction,
        "confidence": round(confidence, 2),
        "department": department,
        "all_detections": detections
    }

```

1.19 10. Gradio Demo (Optional Visualization)

Main file: `model/app.py`

This is a separate demo UI for manual testing: - user uploads an image - YOLO detects objects - the code calls `r.plot()` to draw bounding boxes - it returns: - plotted image - a textual summary like: - Total of 3 issues detected: 2 potholes and 1 garbage detected.

It also exposes two sliders: - confidence threshold (`conf`) - IoU threshold (`iou`)

Gradio launch command: - `python app.py`

1.20 11. Backend System (Express + MongoDB)

Your backend is located here: - `frontback_end/backend/`

1.20.1 11.1 Backend entry point

Main file: `frontback_end/backend/src/index.js`

This file does: 1. Loads env vars (`dotenv.config()`) 2. Connects to MongoDB (`connectDatabase`) 3. Sets CORS rules using `allowedOrigins` 4. Adds security middleware: - `helmet` - `express-mongo-sanitize` 5. Serves uploaded files: - `/uploads` (static) - `/uploads/avatars` (avatar static) 6. Registers routes: - `auth` - `issues` - `comments` - `attachments` - `users` - `votes` 7. Adds Swagger UI at `/api/docs` 8. Adds health check `/api/health`

1.20.1.1 Key code (backend detection flow)

The backend is split by routes, but `/detect` integration happens in: - `frontback_end/backend/src/routes/issues.js` (see section 12)

1.20.2 11.2 Security, CORS, and API Documentation (What the backend does)

Your backend includes important production-style setup:

1. **CORS (Cross-Origin Resource Sharing)**
 - The code allows requests from:
 - configured frontend URLs (`FRONTEND_URL`, `ADMIN_URL`)
 - local dev URLs like `localhost:5173` / `localhost:3000`
 - Requests from other origins are blocked with an error.
2. **Security headers**
 - Uses `helmet()` to set safer HTTP headers.
3. **MongoDB injection protection**
 - Uses `express-mongo-sanitize()` to reduce risks from operator injection.
4. **Swagger API docs**
 - Uses `swagger-jsdoc` and `swagger-ui-express`.
 - Swagger endpoint:
 - `GET /api/docs`
5. **Health check**
 - Endpoint:

- GET /api/health
 - This helps you verify the server is alive and (depending on environment) whether DB connection and services are ready.

1.21 12. Backend Routes (APIs) - Full Explanation

Base URLs in your app: - /api/auth/... - /api/issues... - /api/attachments... - /api/users...

1.21.1 12.1 Authentication APIs

Routes: frontback_end/backend/src/routes/auth.js

1.21.1.1 POST /api/auth/register

Validates request with Joi: - name - email - password - optional role/phone/department/employeeId

Stores user in MongoDB with: - passwordHash (bcrypt hash) - role defaults to user

Returns: - token (JWT) - user (without passwordHash)

1.21.1.2 POST /api/auth/login

Validates: - email - password

Checks: - user exists - bcrypt password matches

Returns: - JWT token + user details

1.21.1.3 GET /api/auth/me

Uses JWT from Authorization: Bearer <token> Returns the current user (passwordHash removed)

1.21.1.4 PUT /api/auth/profile

Uses: - auth middleware (JWT verification) - multer upload of avatar file

Updates: - name, phone - avatarUrl derived from uploaded file path

1.21.1.5 GET /api/auth/Leaderboard

Returns top 10 users by points where role is user

1.21.2 12.2 JWT Middleware

File: frontback_end/backend/src/middleware/auth.js

It checks: - req.headers.authorization exists and starts with Bearer - verifies token with jwt.verify(token, JWT_SECRET) - finds user by payload.userId - attaches user to req.user

1.21.3 12.3 Issue APIs (Citizen reporting + listing + AI detect)

Routes file: frontback_end/backend/src/routes/issues.js

1.21.3.1 12.3.1 POST /api/issues (Create issue)

This route is protected: - auth middleware required - image handled by multer with field name image

Important implementation details: - If location is a string, it tries JSON.parse(req.body.location) - Validates request using createIssueSchema - Creates a new Issue document: - title, description, category, priority - reporterId = req.user._id - image = req.file.path (if image exists) - labels from request (if present) - location saved as GeoJSON Point with coordinates [lng, lat]

Points system: - awards 10 points to the reporter (\$inc: { points: 10 })

1.21.3.2 Key code example (Create issue)

When the citizen submits the issue, the backend: 1. validates request body using Joi 2. stores image using multer (upload.single("image")) 3. saves the issue in MongoDB 4. awards points to the reporter

```
router.post('/', auth, upload.single("image"), async (req, res, next) => {
  const { error, value } = createIssueSchema.validate(req.body, { stripUnknown: true });
  if (error) return res.status(400).json({ message: error.details[0].message });
```

```
  const { title, description, category, priority, location, labels } = value;
  const issue = new Issue({
    title,
    description,
    category,
    priority: priority || "medium",
    reporterId: req.user._id,
    image: req.file ? req.file.path : undefined,
    labels: labels || [],
    location: location
      ? { type: "Point", coordinates: location.coordinates }
      : undefined
  });
```

```
  await issue.save();
```

```
// Award 10 points system
await User.findByIdAndUpdate(req.user._id, { $inc: { points: 10 } });
res.status(201).json(issue);
});
```

1.21.3.3 12.3.2 POST /api/issues/detect (AI detection for uploaded image)

This route: - is protected by auth - uses multer to store uploaded image temporarily - calls the AI server: - `axios.post(`${aiServerUrl}/detect`, form, ...)`

Returns prediction details: - prediction - confidence - department - all_detections

This is the route the citizen UI calls after selecting/capturing an image.

1.21.3.4 Key code example (Backend calls AI server)

```
router.post('/detect', auth, upload.single("image"), async (req, res) => {
  const imagePath = req.file.path;
  const form = new FormData();
  form.append("file", fs.createReadStream(imagePath));

  const aiServerUrl = process.env.AI_SERVER_URL || "http://127.0.0.1:8000";
  const aiResponse = await axios.post(
    `${aiServerUrl}/detect`,
    form,
    { headers: form.getHeaders(), timeout: 90000 }
  );

  const aiData = aiResponse.data;
  res.json({
    prediction: aiData.prediction,
    confidence: aiData.confidence,
    department: aiData.department,
    all_detections: aiData.all_detections
  });
});
```

1.21.3.5 12.3.3 GET /api/issues (List issues)

Supports query params: - page, limit - status, priority, category - search (searches title and description with regex)

It returns: - issues - page, limit, total

Each issue is populated with: - reporterId (name, email, phone) - attachments

1.21.3.6 12.3.4 GET /api/issues/:id

Returns a single issue by id with reporter and attachments populated.

1.21.3.7 12.3.5 PUT /api/issues/:id (Update issue)

Protected by auth Authorization: - only the reporter (same reporterId) OR admin can update

Valid fields: - title, description, category, priority, status, location

It uses updateIssueSchema for validation.

1.21.3.8 12.3.6 DELETE /api/issues/:id

Protected by auth Authorization: - only the reporter OR admin can delete

Deletes related data: - attachments for the issue - comments for the issue - the issue itself

1.22 13. Backend Comment APIs

Routes file: frontback_end/backend/src/routes/comments.js

1.22.0.1 POST /api/issues/:id/comments

- protected by auth
- validates body with createCommentSchema
- creates Comment document:
 - issueId
 - authorId = req.user._id
 - body

1.22.0.2 GET /api/issues/:id/comments

- returns comments sorted by createdAt ascending
 - populates author information
-

1.23 14. Backend Voting APIs

Routes file: frontback_end/backend/src/routes/votes.js

1.23.0.1 POST /api/issues/:id/vote

- protected by auth
- prevents duplicate voting:
 - checks if a Vote exists for (issueId, userId)
 - if exists, returns error
- otherwise creates Vote and increments issue.votes

1.23.0.2 DELETE /api/issues/:id/vote

- removes vote and decrements issue votes

1.24 15. Backend Attachments APIs (Upload images/files)

Routes: - frontback_end/backend/src/routes/attachments.js -
frontback_end/backend/src/routes/attachmentSigned.js

1.24.1 15.1 POST /api/issues/:id/attachments

- protected by auth
- uses multer with 2 modes:
 1. **Local disk mode** (default):
 - file stored in UPLOAD_DIR (or frontback_end/backend/uploads)
 - url returned: /uploads/<filename>
 2. **S3 mode** (if USE_S3=true):
 - multer uses memoryStorage
 - uploads buffer to S3
 - generates signed URL for immediate access

Saves metadata in MongoDB: - Attachment document: - issued - filename - url - s3Key (if S3) - mimeType - uploadedBy

1.24.2 15.2 GET /api/attachments/:id/signed-url

Protected by auth. If S3 is enabled: - generates a fresh signed URL using AWS SDK If not: - returns local /uploads/<filename> URL

Note for your report: - Your current **React UI does not seem to call** these attachment endpoints. - The backend supports it and the admin mock helper has upload functions, but attachments UI wiring is not clearly connected in the shown components.

1.25 16. MongoDB Data Models

Models are under: frontback_end/backend/src/models/

1.25.1 16.1 User

Fields: - name, email (unique + index), passwordHash - role: user or admin - status: active/suspended/inactive (default active) - department, employeeId, phone, avatarUrl - points number for leaderboard

1.25.2 16.2 Issue

Fields: - title, description - reporterId ref to User - status: reported/acknowledged/in-progress/resolved/escalated/closed/open - category, department - priority:

low/medium/high/urgent - image: local image path string - issueType: pothole/garbage/unknown (currently not strongly used by endpoints) - location: GeoJSON Point: - coordinates stored as [lng, lat] - attachments: array of Attachment ids (populated in listing) - labels: array of detected labels from AI - votes: numeric

Also: - `issueSchema.index({ location: "2dsphere" })` for Geo queries

1.25.3 16.3 Comment

Fields: - `issueId` - `authorId` - `body` - `createdAt` timestamp

1.25.4 16.4 Vote

Fields: - `issueId`, `userId`

Unique index: - ensures a user votes only once per issue

1.25.5 16.5 Attachment

Fields: - `issueId` - `filename`, `url` - `s3Key` (optional) - `mimeType` - `uploadedBy`

1.25.6 16.6 IssueHistory

Created as an audit/history schema but the routes shown do not actively write history entries.

1.26 17. Backend Validation and Error Handling

1.26.1 17.1 Joi validation

Validators are in: - `frontback_end/backend/src/validators/`

They define schemas for: - `auth register/login` - `issue create/update` - `comment create`

1.26.2 17.2 Error handling middleware

File: - `frontback_end/backend/src/middleware/errorHandler.js`

It standardizes error output: - logs errors to console - sends { `message` } JSON response with `err.status` or 500

1.27 18. Scripts and Tests

1.27.1 18.1 Seed script

File: - `frontback_end/backend/src/scripts/seed.js`

It: - connects to DB - deletes existing Users (all), Issues, Comments, Attachments - creates:
- one admin (email admin@civicfix.local, password adminpass) - many citizen users -
creates sample issues with random statuses/categories/priority - sometimes creates
comments

1.27.2 18.2 Clear script

File: - frontback_end/backend/src/scripts/clear_db.js

It: - clears everything except admin users - deletes IssueHistory too

1.27.3 18.3 Test

File: - frontback_end/backend/src/__tests__/health.test.js

It checks that /api/health returns { status: 'ok' } (basic)

1.28 19. Frontend - Citizen Portal (Home)

Folder: - frontback_end/frontend/home/

1.28.1 19.1 Key idea

Citizen portal allows: - login/register - report issues with image + geo-location - run AI
detection automatically - view issues list and vote - view and edit profile

1.28.2 19.2 Token handling

File: - frontback_end/frontend/home/src/utils/api.ts

It stores tokens in localStorage under: - civicfix_user_token

API_BASE uses: - import.meta.env.VITE_API_URL or fallback
https://snap2solve.onrender.com/api

1.28.3 19.3 Main app flow

File: - frontback_end/frontend/home/src/App.tsx

What App does: - fetches leaderboard (/auth/leaderboard) - fetches issues repeatedly
every 5 seconds (/issues) - checks authentication using /auth/me - provides navigation
between pages: - home - report - issues - profile

1.28.3.1 AI + issue creation flow (important)

The citizen image flow is inside: -

frontback_end/frontend/home/src/components/ReportIssue.tsx

The steps are: 1. User selects/captures an image 2. UI uploads it to backend for AI: - POST /issues/detect 3. Backend calls AI server and returns predictions 4. UI uses processAiResult to select category automatically based on AI label 5. User submits issue: - UI calls createIssue (passed from App) - POST /issues with multipart form: - title, description, category - image - labels[] (from AI detections) - location as GeoJSON JSON string (Point)

1.28.4 19.4 Citizen reporting UI (code walkthrough)

Main file: - frontback_end/frontend/home/src/components/ReportIssue.tsx

Key behaviors: - Allows: - upload file (<input type="file">) - capture live camera photo (using getUserMedia) - optional "instant capture" - Gets geolocation (using navigator.geolocation) - Calls AI detection using: - fetch(`\${API\_BASE}/issues/detect`, { method: "POST", body: FormData }) - Validates AI result: - if aiAnalysis.all_detections is empty, it blocks submission - but if AI fails, it allows manual submission (per code comment)

1.28.4.1 Key code (report flow)

```
const runAiDetection = async (file: File) => {
  const formData = new FormData();
  formData.append("image", file);

  const res = await fetch(`${API_BASE}/issues/detect`, {
    method: "POST",
    headers: { Authorization: `Bearer ${getToken()}` },
    body: formData
  });

  const data = await res.json();
  setAiAnalysis(data);
  processAiResult(data);
  return data;
};

const handleSubmit = async () => {
  if (aiAnalysis && (!aiAnalysis.all_detections || aiAnalysis.all_detections.length === 0)) {
    alert("This does not appear to be a valid civic issue...");
    return;
  }

  const title = categories.find(c => c.id === selectedCategory)?.name || "Civic Issue";
  const labels = aiAnalysis?.all_detections?.map((d: any) => d.label) || [];

  const tokenId = await onSubmit(title, description, selectedCategory, selectedImage, coordinates, labels);
}
```

```

    if (tokenId) setShowSuccessModal(true);
  };

```

1.28.4.2 *Key code example (Citizen creates issue request)*

This createIssue logic builds a FormData request that matches backend multer + Joi expectations:

```

const createIssue = async (
  title: string,
  description: string,
  category: string,
  file: File,
  coords: { lat: number; lng: number } | null,
  labels?: string[]
) => {
  const formData = new FormData();
  formData.append("title", title);
  formData.append("description", description);
  formData.append("category", category);
  formData.append("image", file); // backend expects field name "image"

  if (labels && labels.length > 0) {
    labels.forEach(label => formData.append("labels[]", label));
  }

  if (locationData) {
    formData.append("location", JSON.stringify({
      type: "Point",
      coordinates: [locationData.lng, locationData.lat]
    }));
  }

  const res = await fetch(`${API_BASE}/issues`, {
    method: "POST",
    headers: { Authorization: `Bearer ${getToken()}` },
    body: formData
  });

  if (res.ok) return await res.json();
  throw new Error((await res.json()).message || "Failed to create issue");
};

```

1.28.5 19.5 Issues list and voting

File: - frontback_end/frontend/home/src/components/IssueListPage.tsx

It supports: - searching - filtering by category/status - sorting by priority/votes/date - voting button that calls onVote(issueId) provided from App

Voting hits: - backend POST /issues/:id/vote

1.28.6 19.6 Profile editing

File: - frontback_end/frontend/home/src/components/ProfilePage.tsx

It allows: - edit name and phone - upload avatar file - calls: - PUT /auth/profile

1.29 20. Frontend - Admin Portal

Folder: - frontback_end/frontend/admin/

1.29.1 20.1 Admin authentication

Main file: - frontback_end/frontend/admin/src/components/AuthForm.tsx

It supports: - login mode - register mode (creates admin user in UI; backend allows role field but backend validation allows role = 'user' or 'admin')

1.29.2 20.2 Admin dashboard

Main UI: - frontback_end/frontend/admin/src/components/AdminDashboard.tsx

It: - loads issues, users, analytics - computes chart data - updates issue status via updateIssue - deletes issues via deleteIssue - supports CSV download generation (Excel and PDF are placeholders in code)

1.29.3 20.3 Issue map (admin)

File: - frontback_end/frontend/admin/src/components/IssueMap.tsx

It shows: - issue quick list - selected issue location preview using Google Maps embed - status update dropdown and delete button

1.29.4 20.4 Issue detail modal (admin)

File: - frontback_end/frontend/admin/src/components/IssueDetailModal.tsx

It: - shows images, description, location, reporter details - fetches comments using: - fetchComments(issue.id) from admin api helpers - allows admin to update status: - updateIssue(issue.id, { status: newStatus })

1.29.4.1 Key code (status update + comments)

```
const handleStatusChange = async (newStatus: string) => {
  setUpdatingStatus(true);
  try {
    await updateIssue(issue.id, { status: newStatus });
    toast.success(`Status → ${newStatus}`);
    if (onUpdate) onUpdate();
  }
}
```

```

    } catch {
      toast.error('Failed to update status');
    } finally {
      setUpdatingStatus(false);
    }
  };

useEffect(() => {
  if (!issue) return;
  fetchComments(issue.id)
    .then((c: any) => setComments(Array.isArray(c) ? c : []))
    .catch(() => {});
}, [issue?.id]);

```

1.30 21. Deployment / Running the Whole System

1.30.1 21.1 Local setup (high-level)

You have 3 parts to run locally: 1. AI server (FastAPI) 2. Backend server (Express + MongoDB) 3. Frontends (React + Vite)

1.30.1.1 21.1.1 Run the AI server (FastAPI)

Folder: `model/`

1. Install dependencies:
 - `pip install -r requirements.txt`
2. Ensure your trained weights exist:
 - Your code expects `model/best.pt` (see `model/api.py` and `model/app.py`)
3. Start FastAPI:
 - `uvicorn api:app --host 0.0.0.0 --port 8000`

Health check: - GET `http://localhost:8000/health`

1.30.1.2 21.1.2 Run the backend (Express + MongoDB)

Folder: `frontback_end/backend/`

1. Install dependencies:
 - `npm install`
2. Create environment variables:
 - The backend uses (from `frontback_end/backend/README.md` and code):
 - `MONGODB_URI` (required)
 - `JWT_SECRET` (required)
 - `PORT` (default: 5000)
 - `UPLOAD_DIR` (optional; default path is used if missing)

- AI_SERVER_URL (optional; default in code is `http://127.0.0.1:8000`)
- Optional for S3:
 - USE_S3=true
 - AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION
 - AWS_S3_BUCKET
 - SIGNED_URL_EXPIRES

3. Start server in dev mode:

- `npm run dev`

Health check: - GET `http://localhost:5000/api/health` (or your configured PORT)

1.30.1.3 21.1.3 Run the frontends (Citizen + Admin)

Both frontends use Vite and read the API base URL from env: - Citizen:

`frontback_end/frontend/home/` - Admin: `frontback_end/frontend/admin/`

Common steps: 1. In each folder, create a `.env` file (or use your Render env) with: -
`VITE_API_URL=http://localhost:<BACKEND_PORT>/api` 2. Install and run: - `npm install - npm run dev`

Optional: run all together (convenient for demos) - From `frontback_end/` you can use: -
`npm install - npm run dev` (it starts backend + both frontends concurrently)

This repository also supports deployment through Render using `render.yaml`.

1.30.2 21.2 Render deployment (as defined in repo)

File: - `render.yaml`

It defines: 1. `snap2solve-ai` (Python, starts `uvicorn api:app`) 2. `snap2solve-backend` (Node, starts `npm start`) 3. `snap2solve-home` (static build) 4. `snap2solve-admin` (static build)

Environment variables passed: - AI_SERVER_URL to backend - FRONTEND_URL and MONGODB_URI and JWT_SECRET - VITE_API_URL to both frontends

1.30.3 21.3 Backend integration guide (already present in repo)

The repository also includes a step-by-step integration document: -
`frontback_end/docs/Backend_Integration.rtf`

This guide describes (in simple English): 1. The backend API base URL used by the frontend (example: `http://localhost:5000/api`) 2. The major endpoints your UI calls: -
`/api/auth/login`, `/api/auth/register`, `/api/auth/me` - `/api/issues` for issue listing and creation - `/api/issues/:id/comments` for comments - `/api/issues/:id/attachments` for uploads - `/api/attachments/:id/signed-url` for signed URL generation 3. The image upload pipeline: - frontend sends `multipart/form-data` to `/api/issues/:id/attachments`

- backend stores file either locally or in S3 - backend saves metadata in MongoDB and returns URL/signed URL

You can cite this file in your report as an “integration explanation reference”.

1.31 22. Expected Workflow (From User Perspective)

This section should help you explain the project “from start to end” in your viva.

1.31.1 Step A: User Registration/Login

1. Open citizen portal
2. Click “Create Account” / “Sign in”
3. App calls:
 - POST /api/auth/register or POST /api/auth/login
4. Backend verifies credentials and returns JWT
5. Frontend stores token in localStorage

1.31.2 Step B: Reporting an Issue (with AI detection)

1. User clicks “Report Issue”
2. User uploads or captures an image
3. UI calls:
 - POST /api/issues/detect
4. Backend calls AI server:
 - POST /detect
5. AI server returns prediction and detection labels
6. UI uses labels to set issue category and passes labels during submission
7. UI calls:
 - POST /api/issues (multipart upload)
8. Backend stores issue in MongoDB and awards 10 points

1.31.3 Step C: Viewing Issues and Voting

1. UI fetches issues:
 - GET /api/issues
2. UI shows list/grid
3. User can vote:
 - POST /api/issues/:id/vote

1.31.4 Step D: Admin Management

1. Admin logs into admin portal
2. Admin dashboard fetches issues and analytics:
 - GET /api/issues

- (analytics endpoint may be mocked/fallback in admin UI)
 - 3. Admin opens issue details:
 - fetches comments
 - 4. Admin updates status:
 - PUT /api/issues/:id
 - 5. Admin can delete issues:
 - DELETE /api/issues/:id
-

1.32 23. Limitations (Based on Current Code)

To keep your report honest, here are limitations that are visible in the code:

1. **Priority selection in citizen UI is not sent to backend create endpoint**
(ReportIssue.tsx sets priority state but createIssue() in App.tsx does not append priority to FormData).
 - Backend therefore uses default priority from Joi schema (medium).
 2. **Issue status strings displayed in some UI parts may be inconsistent** (home admin dashboard uses legacy labels like Pending, In Progress, etc; while backend uses reported, acknowledged, in-progress, etc).
 3. Attachments APIs exist in backend, but the main visible React UI does not clearly use them for uploading extra attachments.
 4. IssueHistory model exists but routes to record history changes are not clearly implemented in visible code.
-

1.33 24. Conclusion

This project successfully integrates: - a deep learning model (YOLOv8) for smart-city issue detection, - an AI inference server (FastAPI), - a REST backend (Express + MongoDB + JWT + uploads), - and two web frontends (citizen + admin),

to create an end-to-end workflow for identifying and reporting civic problems.

1.34 25. Future Scope

1. Real-time video/CCTV inference instead of single image upload.
2. Strong geospatial dashboard with clustering/heatmaps using the location index in MongoDB.
3. Fully connect attachments UI so multiple images can be uploaded to a single issue.
4. Add issue history logging (IssueHistory) for audit trail.
5. Add better analytics endpoints on backend instead of computing/fallback in frontend.

1.35 References

1. Ultralytics YOLOv8 documentation
 2. FastAPI documentation (File upload, CORS, Lifespan)
 3. Express.js documentation
 4. MongoDB + Mongoose documentation
 5. React + Vite documentation
 6. Roboflow dataset annotation platform (as used in project workflow)
-

1.36 Appendix A: Important Code Files

These are the most important “core” files that you can reference directly in your report:

AI: - `model/api.py` (FastAPI inference server) - `model/app.py` (Gradio demo) -
`model/process_local_datasets.py` (dataset merge + class remapping)

Backend: - `frontback_end/backend/src/index.js` (server setup) -
`frontback_end/backend/src/middleware/auth.js` (JWT middleware) -
`frontback_end/backend/src/routes/auth.js` (register/login/me/profile/leaderboard) -
`frontback_end/backend/src/routes/issues.js` (create/list/update/delete/detect) -
`frontback_end/backend/src/routes/comments.js` (comment APIs) -
`frontback_end/backend/src/routes/votes.js` (vote APIs) -
`frontback_end/backend/src/routes/attachments.js` and `attachmentSigned.js` (upload support) - `frontback_end/backend/src/models/*` (User, Issue, Comment, Vote, Attachment, IssueHistory)

Frontends: - `frontback_end/frontend/home/src/App.tsx` (citizen flow) -
`frontback_end/frontend/home/src/components/ReportIssue.tsx` (AI detect + submit issue) - `frontback_end/frontend/admin/src/App.tsx` (admin flow) -
`frontback_end/frontend/admin/src/components/AdminDashboard.tsx` (admin dashboard) - `frontback_end/frontend/admin/src/components/IssueDetailModal.tsx` (issue view + status update + comments)